

Trabalho Prático: Teoria dos Grafos e Computabilidade

Arthur L. Panzera¹, Erick G. Carvalho¹,
João Pedro Lezsi Fernandes¹, Luiz Gustavo Fagundes Teixeira¹

¹ Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Instituto de Ciências Exatas e Informática (ICEI)
Belo Horizonte – MG – Brasil

alpanzera@sga.pucminas.br, erick.carvalho.1450737@sga.pucminas.br,
joao.fernandes.1425541@sga.pucminas.br, luiz.teixeira@sga.pucminas.br

Abstract. *This work presents the development of a computational tool based on Graph Theory to analyze interactions among contributors of a large public GitHub repository. The selected repository was Starship, which satisfies the requirement of having more than 5,000 stars and an active open-source community. Data were collected from GitHub interactions, including issue and pull request comments, issue closing events, pull request reviews, approvals, and merges. These interactions were modeled as directed and weighted graphs, in which users are represented as vertices and interactions as edges. The tool implements graph structures from scratch using adjacency matrix and adjacency list representations, builds separate graphs for different interaction types, and generates an integrated weighted graph. Network analysis metrics such as degree centrality, betweenness, closeness, PageRank, density, clustering coefficient, assortativity, community detection, and bridging ties were applied. The results revealed a sparse and disassortative collaboration network, with a small group of users occupying central positions in the project dynamics.*

Resumo. *Este trabalho apresenta o desenvolvimento de uma ferramenta computacional baseada em Teoria dos Grafos para analisar interações entre colaboradores de um repositório público de grande porte do GitHub. O repositório selecionado foi o Starship, que atende ao requisito de possuir mais de 5.000 estrelas e apresenta uma comunidade ativa de código aberto. Os dados foram coletados a partir de interações do GitHub, incluindo comentários em issues e pull requests, fechamentos de issues, revisões, aprovações e merges de pull requests. Essas interações foram modeladas como grafos direcionados e ponderados, nos quais os usuários são representados como vértices e as interações como arestas. A ferramenta implementa estruturas de grafos do zero, utilizando matriz e lista de adjacência, constrói grafos separados por tipo de interação e gera um grafo integrado ponderado. Foram aplicadas métricas de análise de redes, como centralidade de grau, intermediação, proximidade, PageRank, densidade, coeficiente de agrupamento, assortatividade, detecção de comunidades e bridging ties. Os resultados indicaram uma rede de colaboração esparsa e disassortativa, com um pequeno grupo de usuários ocupando posições centrais na dinâmica do projeto.*

1. Introdução

Este trabalho aplica conceitos da Teoria dos Grafos para analisar a rede de colaboração entre usuários de um repositório público do GitHub com mais de 5.000 estrelas. A ferramenta desenvolvida coleta dados de interações — como comentários em issues e pull requests, fechamento de issues, revisões, aprovações e merges — e os organiza em grafos direcionados e ponderados. Nessa modelagem, cada usuário é representado como um vértice e cada interação como uma aresta, com peso proporcional à relevância da colaboração. A partir dessa estrutura, são calculadas métricas de centralidade, densidade e detecção de comunidades, permitindo identificar colaboradores influentes, grupos de trabalho e padrões de participação no projeto

2. Justificativa da Escolha do Repositório

O repositório selecionado para o desenvolvimento deste trabalho foi o *Starship*, disponível publicamente no GitHub. O projeto consiste em uma ferramenta para personalização de terminais, compatível com ambientes como Bash, Zsh, Fish e PowerShell. A escolha se justifica por sua elevada relevância na plataforma, com mais de 57 mil estrelas, superando amplamente o requisito mínimo de 5.000 estrelas estabelecido para o trabalho.

Além da popularidade, o *Starship* apresenta uma comunidade ativa e um histórico expressivo de colaboração, com centenas de *issues* abertas e mais de 3.700 *pull requests* encerradas. Esse volume de atividades torna o projeto adequado para a extração de dados relacionados a comentários, revisões, aprovações, fechamentos e *merges*, permitindo construir uma rede de interações suficientemente ampla para a aplicação dos conceitos de Teoria dos Grafos.

3. Coleta e Modelagem de Dados

Esta seção apresenta a abordagem de coleta e modelagem dos dados utilizados na construção dos grafos, incluindo ferramentas, tipos de interação e estrutura de armazenamento.

3.1. Estratégia de Extração

A coleta de dados foi baseada na identificação de interações entre usuários do repositório selecionado, conforme os objetivos do trabalho.

3.2. Ferramentas Utilizadas

A extração foi implementada em Python utilizando a API REST do GitHub [GitHub 2026], encapsulada na classe `GitHubClient`, responsável por requisições, paginação, autenticação e controle de limites.

Para otimização, foi utilizado um sistema de cache local via `CacheJson`, reduzindo chamadas à API em execuções subsequentes. As configurações do minerador são gerenciadas pela classe `MinerConfig`.

A orquestração da coleta de *issues*, *pull requests*, comentários e eventos associados é realizada pela classe `GitHubMinerador`, que padroniza e armazena os dados no arquivo `dados/interacoes.json`. O projeto também conta com suporte a Docker e Docker Compose para facilitar a execução e replicação do ambiente.

3.3. Interações Analisadas e Transformação em Arestas

A coleta de dados terá como foco as interações que evidenciam a colaboração entre os usuários do repositório selecionado. Na modelagem proposta, cada usuário será representado como um vértice, enquanto as ações realizadas entre diferentes colaboradores serão representadas como arestas direcionadas. As interações serão mapeadas da seguinte forma:

- **Comentários em *issues* e *pull requests*:** quando um usuário comentar em uma *issue* ou em um *pull request* criado por outro colaborador, será criada uma aresta direcionada do autor do comentário para o autor da publicação. Assim, se o usuário B comentar em uma contribuição aberta pelo usuário A, será criada a aresta $(B \rightarrow A)$. Caso A posteriormente responda a B, também será criada a aresta $(A \rightarrow B)$.
- **Fechamento de *issues*:** quando uma *issue* aberta pelo usuário A for encerrada pelo usuário B, desde que sejam usuários distintos, será criada uma aresta direcionada de B para A, representada por $(B \rightarrow A)$.
- **Revisões, aprovações e *merges* de *pull requests*:** quando o usuário B revisar, aprovar ou realizar o *merge* de um *pull request* submetido pelo usuário A, será criada uma aresta direcionada de B para A, representada por $(B \rightarrow A)$. Essa relação expressa a participação de B na avaliação ou incorporação da contribuição realizada por A.

O grafo resultante será simples e direcionado, de modo que não serão permitidos laços ou arestas duplicadas. Nos casos em que dois usuários interagirem mutuamente, a relação será representada por arestas antiparalelas, preservando a direção de cada interação.

Tabela 1. Mapeamento das interações em arestas direcionadas

Interação	Origem da aresta	Destino da aresta
Comentário em <i>issue</i> ou <i>pull request</i>	Usuário que realizou o comentário	Autor da <i>issue</i> ou do <i>pull request</i> comentado
Fechamento de <i>issue</i>	Usuário que fechou a <i>issue</i>	Autor da <i>issue</i>
Revisão ou aprovação de <i>pull request</i>	Usuário que revisou ou aprovou o <i>pull request</i>	Autor do <i>pull request</i>
<i>Merge</i> de <i>pull request</i>	Usuário que realizou o <i>merge</i>	Autor do <i>pull request</i>

A Tabela 1 resume a direção adotada para cada tipo de interação, deixando explícito qual usuário será considerado origem e qual será considerado destino na criação das arestas.

Cada aresta direcionada representa uma ação de colaboração realizada pelo usuário de origem sobre uma contribuição associada ao usuário de destino, permitindo identificar como os participantes interagem dentro do repositório.

3.4. Modelagem dos Grafos

Com base nas regras de transformação das interações em arestas apresentadas na Tabela 1, serão construídos grafos simples e direcionados para representar as relações de colaboração entre os usuários do repositório. Todos os grafos utilizarão o mesmo conjunto de vértices, formado pelos usuários identificados na coleta, mas possuirão conjuntos de arestas distintos de acordo com o tipo de interação analisada.

Para possibilitar a análise individual de cada tipo de colaboração, serão construídos três grafos distintos:

- **Grafo de comentários em *issues* e *pull requests*:** reunirá as interações realizadas por meio de comentários em *issues* e *pull requests*. Esse grafo permitirá observar quais usuários participam mais ativamente das discussões do projeto.
- **Grafo de fechamento de *issues*:** reunirá as interações em que uma *issue* aberta por um usuário foi encerrada por outro colaborador. Esse grafo permitirá identificar usuários que atuam na organização e resolução das demandas registradas no repositório.
- **Grafo de revisões, aprovações e *merges* de *pull requests*:** reunirá as interações relacionadas à avaliação e integração de contribuições ao projeto. Esse grafo permitirá analisar a participação dos usuários em atividades de validação e incorporação de código.

Além dos grafos específicos, será construído um **grafo integrado e ponderado**, reunindo as diferentes interações analisadas em uma única estrutura. Nesse grafo, os pesos das arestas serão definidos de acordo com a intensidade e a relevância da colaboração representada, conforme apresentado na Tabela 2.

Tabela 2. Pesos atribuídos às interações no grafo integrado

Tipo de interação	Peso
Comentário em <i>issue</i> ou <i>pull request</i>	2
Abertura de <i>issue</i> comentada por outro usuário	3
Revisão ou aprovação de <i>pull request</i>	4
<i>Merge</i> de <i>pull request</i>	5

Na modelagem proposta, os grafos separados seguem as relações indicadas no enunciado: comentários em *issues* ou *pull requests*, fechamento de *issues* por outro usuário e revisões, aprovações ou *merges* de *pull requests*. Já no grafo integrado, os pesos seguem a sugestão do enunciado para representar a intensidade das interações. Assim, o peso 3 é atribuído à situação em que uma *issue* aberta por um usuário recebe comentário de outro usuário, enquanto o fechamento de *issues* permanece representado no Grafo 2 como uma relação específica de colaboração.

O peso final de uma aresta no grafo integrado será calculado pela seguinte expressão:

$$P(u, v) = 2C(u, v) + 3I(u, v) + 4R(u, v) + 5M(u, v)$$

em que $C(u, v)$ representa a quantidade de comentários classificados com peso 2, como comentários gerais em *issues* ou *pull requests* que não se enquadrem na categoria de

peso 3; $I(u, v)$ representa a quantidade de *issues* abertas por v que receberam comentário de u , classificadas com peso 3; $R(u, v)$ representa a quantidade de revisões ou aprovações feitas por u em *pull requests* criados por v ; e $M(u, v)$ representa a quantidade de *merges* realizados por u em *pull requests* criados por v .

Para evitar contagem duplicada, cada interação coletada será classificada em uma única categoria de peso no momento da construção do grafo integrado.

Por exemplo, se o usuário B comentar uma *issue* aberta pelo usuário A, comentar um *pull request* de A e também aprovar esse *pull request*, a aresta $B \rightarrow A$ terá peso final $3 + 2 + 4 = 9$. Caso o usuário A também interaja com contribuições de B, será criada uma aresta no sentido oposto, $A \rightarrow B$, caracterizando arestas antiparalelas.

A atribuição de pesos permitirá diferenciar interações mais simples, como comentários, de ações que representam maior participação técnica no projeto, como revisões, aprovações e *merges*. Caso dois usuários realizem interações em sentidos opostos, essas relações serão representadas por arestas antiparalelas. Além disso, por se tratar de grafos simples, não serão permitidos laços nem múltiplas arestas entre os mesmos vértices em cada grafo; no grafo integrado, interações repetidas entre os mesmos usuários serão consolidadas pelo acúmulo de seus respectivos pesos.

Com essa modelagem, será possível analisar separadamente os diferentes tipos de colaboração e, ao mesmo tempo, obter uma visão consolidada da rede de interações do repositório selecionado.

3.5. Representação Visual dos Grafos

Após a mineração dos dados e a construção das estruturas de grafos, os grafos gerados foram exportados no formato GEXF, compatível com a ferramenta Gephi. Essa etapa teve como objetivo permitir a visualização das redes de colaboração e facilitar a interpretação visual das relações entre os usuários do repositório analisado.

Foram geradas representações visuais para os grafos correspondentes aos comentários, fechamentos de *issues*, interações em *pull requests* e ao grafo integrado ponderado. Em cada visualização, os vértices representam usuários do repositório e as arestas direcionadas representam interações entre eles. O grafo integrado reúne todos os tipos de interação coletados, permitindo observar uma visão consolidada da rede de colaboração.

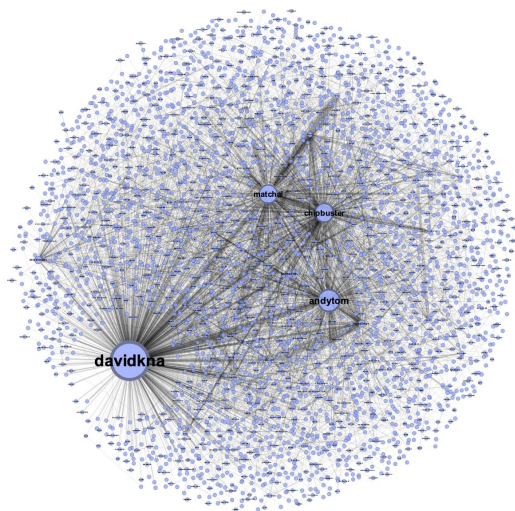


Figura 1. Grafo de comentários em *issues* e *pull requests*.

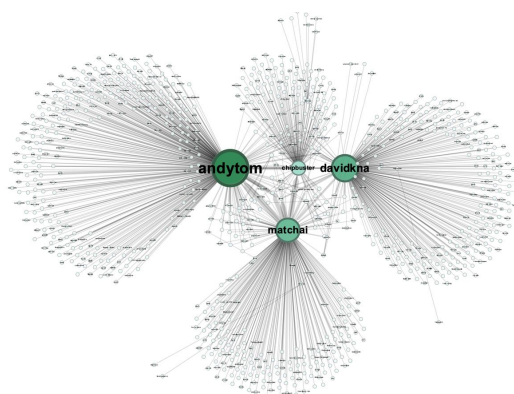


Figura 2. Grafo de fechamento de *issues*.

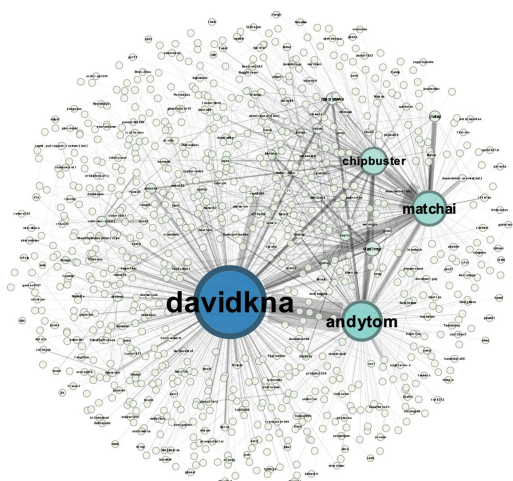


Figura 3. Grafo de interações em *pull requests*.

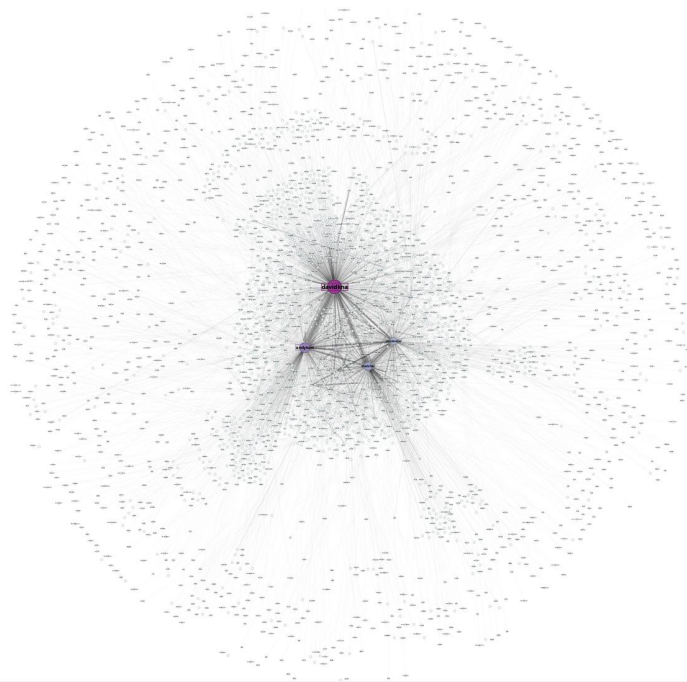


Figura 4. Grafo integrado ponderado das interações coletadas.

A análise visual dos grafos permite observar a presença de usuários com maior concentração de conexões, indicando possíveis colaboradores centrais na dinâmica do projeto. Também é possível perceber a formação de agrupamentos locais, que podem representar subgrupos de colaboração ou usuários que interagem em torno de determinadas *issues* e *pull requests*.

4. Modelagem Técnica da Solução

A solução foi organizada em módulos independentes, responsáveis pela mineração dos dados do GitHub, armazenamento das interações em um formato comum, construção dos grafos, análise das métricas e demonstração da aplicação. Essa separação facilita a manutenção do código e evita o acoplamento direto entre a coleta dos dados e a implementação das estruturas de grafos.

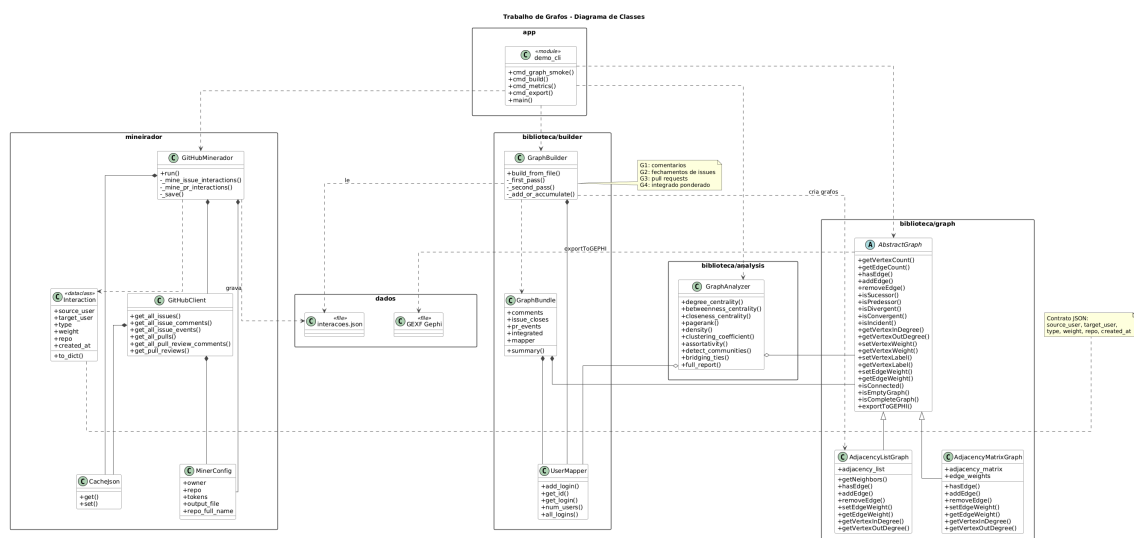


Figura 5. Visão geral da arquitetura da solução.

4.1. Minerador do GitHub

O minerador do GitHub é responsável por consultar os dados públicos do repositório selecionado e transformar os eventos coletados em interações padronizadas entre usuários. Esse módulo acessa informações relacionadas a *issues*, comentários, eventos de fechamento, *pull requests*, comentários em *pull requests*, revisões e informações de *merge*.

A classe principal desse módulo é a `GitHubMinerador`, responsável por coordenar todo o processo de mineração. Ela utiliza a classe `GitHubClient` para realizar as requisições à API REST do GitHub, a classe `MinerConfig` para carregar as configurações do ambiente e a classe `CacheJson` para armazenar localmente respostas já obtidas da API.

A classe `GitHubClient` encapsula a comunicação com a API do GitHub, tratando paginação, autenticação por token, rotação de tokens, limitação de requisições e novas tentativas em caso de erro. Já o `CacheJson` evita chamadas repetidas à API, permitindo reaproveitar dados coletados anteriormente e tornando a execução mais estável durante testes e apresentações.

Cada evento minerado é encapsulado pela classe `Interaction` (contendo origem, destino, tipo, peso, repositório e data) e persistido sequencialmente no arquivo `dados/interacoes.json` em formato JSON Lines.

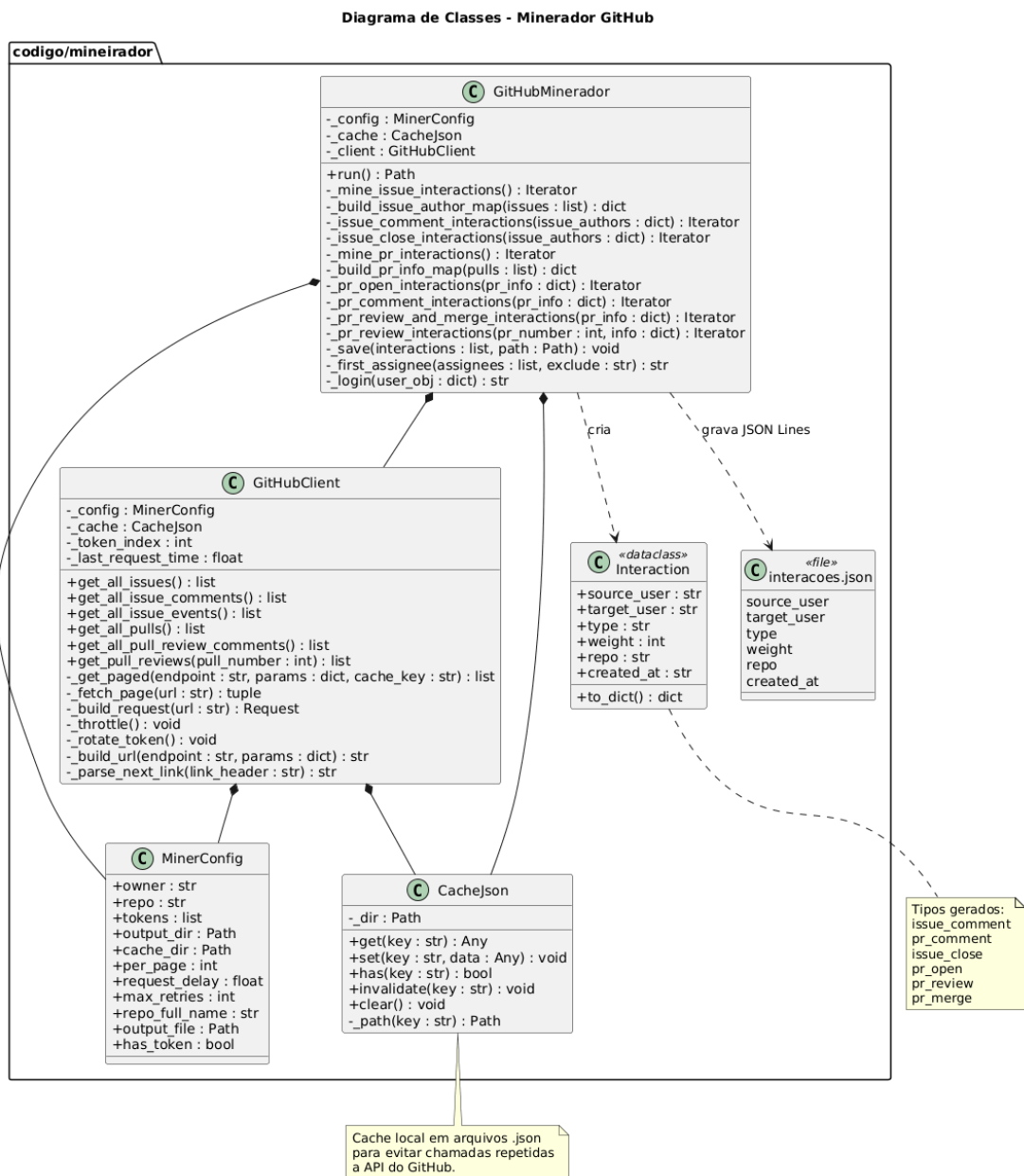


Figura 6. Diagrama de Classes minerador.

4.2. Formato Comum das Interações

Para padronizar o processamento dos eventos do GitHub, todas as interações são convertidas para um contrato JSON único, utilizado como entrada na construção dos grafos.

- **source_user:** usuário que realizou a ação;
- **target_user:** usuário associado ao evento;
- **type:** tipo da interação;
- **weight:** peso da interação;
- **repo:** repositório de origem;
- **created_at:** data do evento.

Os tipos considerados são `issue_comment`, `pr_comment`, `issue_close`, `pr_open`, `pr_review` e `pr_merge`, que definem como as arestas são classificadas no grafo.

4.3. Construção dos Grafos

A construção é realizada pela classe `GraphBuilder`, que lê o arquivo `dados/interacoes.json` e converte as interações em arestas direcionadas. O mapeamento de usuários textuais para índices inteiros é feito pela classe `UserMapper`.

O processo ocorre em duas etapas: identificação dos usuários únicos e inserção das arestas nos grafos correspondentes.

São gerados quatro grafos: comentários, fechamento de *issues*, eventos de *pull requests* e o grafo integrado ponderado. Todos são organizados em um `GraphBundle`.

Os grafos utilizam `AdjacencyListGraph`, mais eficiente para redes esparsas.

4.4. API de Grafos

A API foi implementada do zero. A classe abstrata `AbstractGraph` define operações comuns, como validação, consulta e exportação para GEXF.

Duas implementações foram desenvolvidas: `AdjacencyMatrixGraph` e `AdjacencyListGraph`. A API suporta manipulação de arestas, consulta de graus, pesos e verificação estrutural, respeitando restrições de grafos simples e direcionados.

4.5. Análise dos Grafos

A classe `GraphAnalyzer` calcula métricas de rede no grafo integrado, incluindo centralidades, PageRank, densidade, coeficiente de agrupamento, assortatividade e detecção de comunidades. Os resultados são organizados em rankings para análise da estrutura da rede.

4.6. Demonstração e Exportação

A aplicação de demonstração (`demo_cli.py`) executa testes da API com dados sintéticos e processa os dados reais do minerador. Também realiza a construção e análise dos grafos.

A exportação é feita em formato GEXF via `exportToGEPHI`, permitindo visualização no Gephi para todos os tipos de grafos gerados.

5. Análise de Redes

Esta seção apresenta as métricas aplicadas ao grafo integrado gerado a partir das interações coletadas no repositório *Starship*. A análise foi realizada pela classe `GraphAnalyzer`, implementada sem o uso de bibliotecas prontas de grafos. As métricas permitem identificar usuários centrais, avaliar a coesão da rede e observar a formação de comunidades.

5.1. Métricas de Centralidade

As métricas de centralidade buscam identificar os usuários mais relevantes na rede de colaboração.

- **Grau:** mede a quantidade de conexões diretas de cada usuário. Como o grafo é direcionado, são considerados o grau de entrada e o grau de saída. Essa métrica ajuda a identificar usuários que recebem muitas interações e usuários que interagem frequentemente com contribuições de outros.
- **Intermediação:** mede a frequência com que um usuário aparece em caminhos mínimos entre outros pares de vértices. Essa métrica identifica usuários que atuam como ponte entre diferentes regiões da rede.
- **Proximidade:** mede o quão próximo um usuário está dos demais vértices alcançáveis da rede, permitindo identificar participantes estruturalmente bem posicionados.
- **PageRank:** atribui maior relevância a usuários que recebem interações de outros usuários também importantes dentro da rede, destacando participantes influentes na dinâmica de colaboração. ““

5.2. Métricas de Estrutura e Coesão

As métricas de estrutura e coesão descrevem propriedades globais da rede.

- **Densidade:** mede a proporção de arestas existentes em relação ao número máximo possível de conexões. Valores baixos indicam redes esparsas, característica comum em projetos de código aberto com muitos participantes.
- **Coefficiente de aglomeração:** mede a tendência dos vizinhos de um usuário também estarem conectados entre si, indicando a formação de grupos locais de colaboração.
- **Assortatividade:** mede a tendência de usuários se conectarem a outros com grau semelhante. Valores positivos indicam conexão entre usuários de perfil semelhante, enquanto valores negativos sugerem interações entre usuários centrais e periféricos. ““

5.3. Métricas de Comunidades

A análise de comunidades busca identificar grupos de usuários que interagem com maior frequência entre si.

- **Deteção de comunidades:** foi implementada por propagação de rótulos, técnica que agrupa usuários com padrões de interação semelhantes, permitindo identificar subgrupos de colaboração dentro da rede.
- **Bridging ties:** identifica usuários que conectam comunidades distintas, atuando como pontos de ligação entre diferentes grupos da rede. ““

5.4. Complexidade Computacional

As métricas implementadas apresentam diferentes custos computacionais, influenciados pelo número de vértices (V) e arestas (E) do grafo.

- **Centralidade de grau:** possui complexidade $O(V + E)$.

- **Centralidade de intermediação:** implementada com o algoritmo de Brandes, possui complexidade $O(VE)$ para grafos não ponderados.
- **Centralidade de proximidade:** requer buscas em largura a partir de cada vértice, apresentando complexidade $O(V(V + E))$.
- **PageRank:** é calculado iterativamente até a convergência, apresentando complexidade aproximada de $O(kE)$, em que k representa o número de iterações.
- **Coefficiente de aglomeração:** pode alcançar complexidade $O(Vd^2)$, em que d representa o grau médio dos vértices.
- **Deteção de comunidades:** apresenta complexidade aproximadamente linear $O(E)$ por iteração.

Como redes de colaboração do GitHub tendem a ser esparsas, a utilização de listas de adjacência reduz o consumo de memória e melhora o desempenho das operações quando comparada à representação por matriz de adjacência.

As métricas descritas permitem analisar a rede sob três perspectivas complementares: a importância individual dos usuários, a estrutura geral da rede e a formação de grupos de colaboração.

6. Demonstração do Sistema

Esta seção apresenta as principais telas da aplicação desenvolvida, evidenciando o funcionamento do sistema desde a execução dos comandos principais até a visualização dos resultados obtidos. As imagens ilustram a interface geral do sistema, a construção dos grafos e as representações em lista e matriz utilizadas para análise das redes geradas.

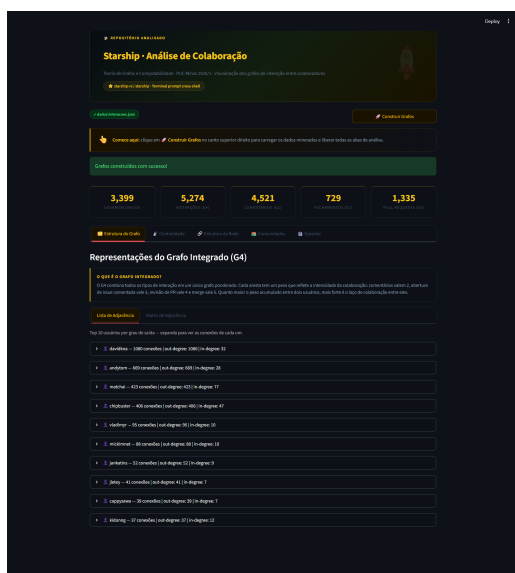


Figura 7. Tela geral do sistema desenvolvida com Streamlit.



Figura 8. Construção dos grafos.

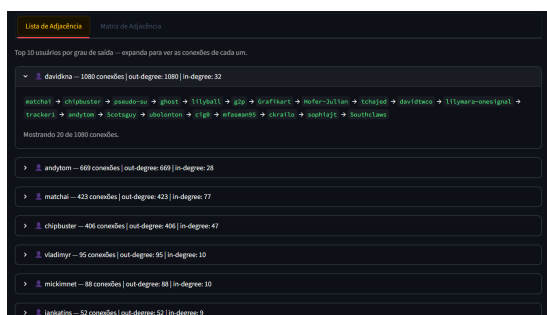


Figura 9. Lista de adjacência.

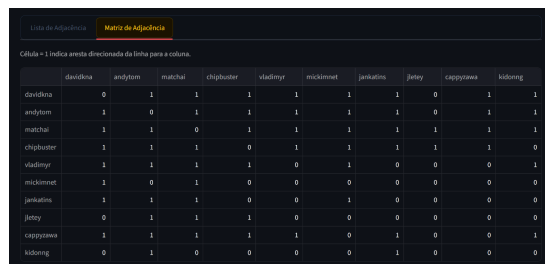


Figura 10. Matriz de adjacência.

As telas demonstram que a aplicação permite executar o fluxo principal do trabalho: coletar os dados do GitHub, construir os grafos e visualizar diferentes representações estruturais das redes geradas.

7. Resultados e Discussão

Esta seção apresenta os resultados da análise do grafo integrado do repositório *Starship*, construído a partir de interações como comentários, *issues*, *pull requests* e *merges*. A análise foi realizada pela classe `GraphAnalyzer`, utilizando métricas de rede complexa.

O grafo possui 3.406 vértices e 5.353 arestas direcionadas, indicando uma rede esparsa. A Tabela 3 apresenta os principais indicadores.

Tabela 3. Indicadores gerais do grafo integrado

Métrica	Valor
Número de vértices	3.406
Número de arestas	5.353
Densidade	0,000462
Assortatividade	-0,2699
Comunidades detectadas	99

A baixa densidade confirma o comportamento esparsa típico de repositórios open source. A assortatividade negativa (-0,2699) indica padrão disassortativo, com usuários altamente conectados interagindo com usuários menos ativos.

7.1. Centralidade

As métricas de centralidade destacam os principais usuários da rede. A Tabela 4 apresenta o ranking por grau total.

Tabela 4. Top 10 usuários por centralidade de grau

Usuário	Entrada	Saída	Total
davidkna	0,0094	0,3166	0,1630
andytom	0,0082	0,2029	0,1056
matchai	0,0226	0,1389	0,0808
chipbuster	0,0138	0,1236	0,0687
vladimyr	0,0029	0,0279	0,0154

Observa-se que davidkna lidera a centralidade de grau, seguido por andytom, matchai e chipbuster. Esses usuários concentram maior volume de interações na rede.

A Tabela 5 mostra a centralidade de intermediação.

Tabela 5. Top 5 usuários por intermediação

Usuário	Intermediação
davidkna	0,029119
matchai	0,026797
chipbuster	0,020060
andytom	0,015513
ghost	0,004933

Os mesmos usuários também aparecem nas principais posições de grau, indicando forte relevância estrutural.

7.2. PageRank e Comunidades

O PageRank reforça que diferentes métricas capturam aspectos distintos da rede. A Tabela 6 mostra os principais valores.

Tabela 6. Top 5 usuários por PageRank

Usuário	PageRank
matchai	0,010818
chipbuster	0,007117
chlopes	0,003916
ghost	0,003585
davidkna	0,003366

A detecção de comunidades identificou 99 grupos, sendo um componente principal dominante com a maioria dos usuários e diversos subgrupos menores. Isso indica forte concentração colaborativa com interações pontuais em nichos específicos.

Também foram identificados usuários que atuam como *bridging ties*, conectando comunidades diferentes, como gmaghera, Vader699 e Mekk.

7.3. Síntese

A rede é esparsa, disassortativa e altamente concentrada em poucos usuários centrais. As métricas de centralidade mostram que diferentes medidas destacam usuários parcialmente distintos, reforçando a importância de múltiplas perspectivas na análise. A estrutura comunitária revela um grande componente principal e diversos grupos menores conectados por nós intermediários.

8. Contribuição dos Membros do Grupo

Todos os membros do grupo participaram ativamente na modelagem e construção dos grafos do projeto (G_1 , G_2 , G_3 e G_4), incluindo a definição das interações, atribuição de pesos e validação das estruturas.

Membro	Principais Contribuições no Projeto
João	Desenvolvimento do módulo de mineração de dados do GitHub, extração/processamento das interações e elaboração dos diagramas.
Luiz	Desenvolvimento do minerador da API do GitHub, integração dos módulos, testes de recolha, análise de resultados e gravação/edição do vídeo demonstrativo.
Erick	Trabalho conjunto na biblioteca: implementação da lista de adjacência da API de grafos, integração do sistema e redação do relatório técnico em $\text{\LaTeX}$.
Arthur	Trabalho conjunto na biblioteca: implementação da matriz de adjacência, testes das operações da API, exportação para o Gephi e revisão técnica.

9. Conclusão

Este trabalho apresentou uma ferramenta em Teoria dos Grafos para analisar a rede de colaboração do repositório *Starship*. O fluxo foi validado com sucesso, abrangendo desde a mineração de dados do GitHub até o cálculo de métricas estruturais e a exportação para o Gephi por meio de estruturas próprias de matriz e lista de adjacência.

Os resultados revelaram uma rede esparsa e disassortativa com 3.406 vértices e 5.353 arestas. Esse padrão indica que as interações estão concentradas em poucos colaboradores centrais — como *davidkna*, *matchai*, *chipbuster* e *andytom* — que atuam como pontes para os participantes periféricos. A análise de comunidades confirmou a existência de um componente principal dominante e subgrupos menores interconectados.

A ferramenta cumpre o objetivo de transformar interações brutas do GitHub em inteligência de rede, fornecendo uma visão estruturada sobre a dinâmica de colaboração em projetos de código aberto.

Referências

GitHub (2026). Github rest api documentation. Acesso em: 12 jun. 2026.